Evaluating Lightpanda Browser Performance and Compatibility in End-To-End Tests

Ing. Andreas Salminger, BSc.
Mobile Computing Master's program
University of Applied Sciences Upper Austria,
Campus Hagenberg

Clemens Zellinger, BSc.
Mobile Computing Master's program
University of Applied Sciences Upper Austria,
Campus Hagenberg

***Abstract*—Lightpanda is a purpose-built headless browser written in Zig that advertises up to 9× faster execution and 16× lower memory consumption compared to Google Chrome, positioning itself as a drop-in replacement for Chrome Headless in end-to-end (E2E) test pipelines. This paper presents an empirical evaluation of this claim across three progressively realistic test stages using Playwright and the Chrome DevTools Protocol (CDP).**

Stage 1 confirms performance advantages on controlled micro-benchmarks: Lightpanda is approximately 2× faster and uses 10× less memory than Chrome against a local web server. Stage 2 reproduces the speedup in a real GitHub Actions CI pipeline running three Playwright tests against TodoMVC, achieving 1.23× faster total job duration and 1.50× faster test execution. However, Stage 3 – a suite of 40 Playwright tests against a realistic Angular-based e-commerce testing application – reveals a critical incompatibility: 30 out of 40 tests fail on Lightpanda, while all 40 pass on Chrome without modification. Failures stem from missing browser APIs required by Playwright, inability to create isolated browser contexts, absent CDP domain support, incomplete JavaScript engine coverage, and missing locale support.

Furthermore, Lightpanda's published benchmark figures are based on a purpose-built trivial demo page rather than real web applications, raising serious concerns about the ecological validity of their advertised performance claims. We conclude that Lightpanda, in its current beta state, is not a production-ready replacement for Google Chrome Headless in real-world E2E test pipelines.

I. Introduction

End-to-end testing is a cornerstone of modern continuous delivery pipelines. By simulating real user interactions in a browser, E2E tests validate application behaviour across the full stack without mocking, providing a high-confidence quality gate before every deployment [1]. Google Chrome in headless mode is the established standard for this purpose: through the Chrome DevTools Protocol (CDP), it is controlled by frameworks such as Playwright [2] without requiring a display. Its principal drawbacks in CI environments are a large binary footprint (exceeding 100 MB) and significant per-instance memory consumption, which translate into longer pipeline durations and higher compute costs on resource-constrained cloud runners [3].

Lightpanda[1] is a recently introduced headless browser implemented in Zig, targeting cloud and CI workloads. Its homepage advertises execution time **9× faster** and memory usage **16× lower** than Chrome, citing a benchmark across 933 "real web pages." It also exposes a CDP endpoint, which in principle allows existing Playwright test suites to connect without modification. These claims motivate a natural question: *Does Lightpanda actually work as a drop-in replacement for Chrome Headless in production E2E test pipelines?*

This paper answers that question through a three-stage empirical study of increasing realism. Stage 1 evaluates raw performance on a controlled local benchmark. Stage 2 exercises Lightpanda in a real GitHub Actions CI pipeline with a simple test application. Stage 3—the most critical stage—runs a comprehensive Playwright suite of 40 tests against a realistic multi-page web application, revealing where Lightpanda's CDP and JavaScript engine coverage breaks down in practice. We additionally examine the validity of Lightpanda's own published benchmark methodology.

The key finding is that while Lightpanda's performance advantages are real and reproducible in synthetic settings, they are rendered moot by severe compatibility failures in real application testing: 30 of 40 tests fail on Lightpanda while passing on Chrome. Lightpanda is not yet production-ready as a Chrome Headless replacement.

II. Background

A. Headless Browsers and the Chrome DevTools Protocol

A headless browser executes JavaScript, renders the DOM, and responds to automation commands without a graphical display. Google Chrome introduced native headless mode in 2017, superseding earlier tools such as PhantomJS [4] and quickly becoming the dominant choice for browser automation due to its comprehensive web standards support. Its primary drawback in CI contexts is resource intensity: Chrome's multi-process architecture spawns dedicated renderer, GPU, and network service processes, resulting in hundreds of megabytes of resident memory per instance [5].

The Chrome DevTools Protocol (CDP) is a WebSocket-based remote control interface that exposes browser internals—DOM manipulation, JavaScript execution, navigation, and network interception—to external automation tools [6]. Because Lightpanda implements a CDP-compatible server, Playwright can connect to it using the same endpoint configuration as for Chrome, making the two engines nominally interchangeable at the protocol level.

[1] https://lightpanda.io

B. Playwright and E2E Testing in CI

Playwright is an open-source browser automation framework by Microsoft that provides high-level APIs for simulating user interactions: clicks, form input, navigation, and DOM assertions [2]. It connects to browsers via CDP and supports *browser contexts*—isolated browsing sessions analogous to incognito windows—for test isolation. Modern E2E test suites rely heavily on these contexts to prevent state leakage between tests and ensure reproducibility [2].

In CI pipelines such as GitHub Actions, E2E tests run on ephemeral cloud runners with fixed resource quotas. Total pipeline duration is dominated not only by test execution but by setup overhead (browser download and initialisation), which Chrome's large binary makes expensive on cold runners [7]. A lightweight alternative that reduces both setup and execution cost would be directly beneficial, provided it preserves compatibility.

C. Lightpanda's Published Benchmark

Lightpanda's homepage claims 9× faster execution and 16× lower memory consumption compared to Chrome, based on a "crawler benchmark" that navigates 933 "real web pages." Inspection of the benchmark repository[2] reveals that these pages are instances of a purpose-built Lightpanda demo application (`demo-browser.lightpanda.io/amiibo/`): a trivial static HTML page consisting of a single image and a small number of internal links. This is not a representative real-world web application. The validity of performance claims derived from such a benchmark is therefore limited, and independent empirical evaluation against real web applications is necessary—which this paper provides.

III. Methodology

All experiments compare Lightpanda (version 0.3.0) against Google Chrome Headless using Playwright connected via CDP. The study is structured in three stages of increasing realism.

A. Stage 1: Local Micro-Benchmark

A Node.js Express server serves a static HTML page locally, eliminating network latency. For each iteration, a Playwright script connects to the running browser via CDP, creates a new isolated browser context, navigates to the local page, and closes the context. Four metrics are recorded: *ConnectionTimeMs* (CDP WebSocket handshake), *NavigationTimeMs* (`page.goto()` completion), *TotalTimeMs* (full iteration wall-clock), and *MemoryUsageMB* (browser process resident memory). 100 iterations per engine are executed on three hardware configurations: an Intel i5-1135G7 laptop (Win11, 16 GB), an Intel i5-13400F desktop (Win11, 16 GB), and an Apple MacBook Pro M3 Max (48 GB). A cold-start variant runs both engines as fresh Docker containers per iteration to measure startup overhead. On Windows hosts, Lightpanda requires Docker or WSL2 as it has no native binary; Chrome also runs in Docker there for a fair comparison.

[2]https://github.com/lightpanda-io/demo/blob/main/BENCHMARKS.md

TABLE I
STAGE 1 – LOCAL CONNECT-MODE BENCHMARK (100 ITERATIONS, MEAN VALUES)

Host	Metric	Lightpanda	Chrome	Chrome Δ%
Windows 11, i5-13400F	Total time (ms)	30.0	59.4	+98%
	Navigation (ms)	10.0	21.2	+112%
	Connection (ms)	13.2	8.9	−32%
	Memory (MB)	6.2	69.8	+1006%
MacBook M3 Max	Total time (ms)	25.6	52.6	+105%
	Navigation (ms)	8.6	16.6	+93%
	Connection (ms)	11.6	10.3	−11%
	Memory (MB)	28.9	274.6	+851%

B. Stage 2: GitHub Actions CI Pipeline

Two GitHub Actions workflows—one per engine—run three Playwright tests against the TodoMVC React demo.[3] The tests verify adding, completing, and filtering todo items. The Chrome workflow installs Chromium via `npx playwright install`; the Lightpanda workflow downloads a single binary (˜20 MB). Both run on `ubuntu-latest` and use an identical test file, differing only in the `CDP_ENDPOINT` variable. Pipeline timing (setup overhead, test execution, teardown, total duration) is recorded over 10 runs per engine.

C. Stage 3: Real Application Test

Stage 3 is the most practically relevant experiment. A suite of 40 Playwright tests is executed against `practicesoftwaretesting.com`, a publicly available Angular-based e-commerce application designed for testing practice. The suite covers five functional areas: product listing (pagination, sorting, filtering, search), product detail pages, cart and checkout access, authentication (login, register, forgot password), and a contact form. Tests include simple visibility assertions, multi-step user flows (login then navigate), DOM content verification, and URL-based navigation assertions. All tests pass on Google Chrome. The same test file is run against Lightpanda without modification, and all failures are recorded along with their root causes.

IV. Results

A. Stage 1: Local Micro-Benchmark

Table I summarises mean performance across 100 iterations in connect mode on two representative platforms. Lightpanda is approximately 2× faster in total request time and uses 10–16× less memory across all configurations. The single phase where Chrome leads is the initial CDP WebSocket handshake, which Chrome completes 11–32% faster than Lightpanda – likely reflecting the maturity gap between Chrome's well-optimised DevTools server and Lightpanda's early-stage CDP implementation. In the cold-start (launch) mode, Lightpanda starts as a Docker container 33–98% faster than Chrome and remains 1.4–1.6× faster overall.

The performance advantage is concentrated in the navigation phase, where Lightpanda's minimal rendering pipeline

[3]https://todomvc.com/examples/react/dist/

TABLE II
STAGE 2 – GITHUB ACTIONS PIPELINE METRICS (10 RUNS, MEAN VALUES)

Metric	Lightpanda	Chrome	Chrome Δ%
Total duration (s)	18.2	22.3	+22.5%
Setup overhead (s)	12.6	15.2	+20.6%
Test execution (s)	3.0	4.5	+50.0%
Teardown (s)	0.5	0.4	n/a

TABLE III
STAGE 3 – TEST RESULTS ON PRACTICESOFTWARETESTING.COM (40 TESTS)

Category	Chrome	LP pass	LP pass rate
Product listing (10 tests)	10/10	1/10	10%
Product detail (9 tests)	9/9	0/9	0%
Cart / checkout (5 tests)	5/5	0/5	0%
Authentication (8 tests)	8/8	7/8	88%
Contact form (8 tests)	8/8	2/8	25%
Total	**40/40**	**10/40**	**25%**

avoids the layout engine, CSS cascade, and compositor overhead that Chrome executes even in headless mode. Memory consumption differs structurally: Chrome's multi-process architecture maintains dedicated renderer, GPU, and network service processes [5], whereas Lightpanda runs as a single lightweight process with no display subsystem.

B. Stage 2: GitHub Actions CI Pipeline

Table II shows mean pipeline metrics over 10 runs per engine. Lightpanda completes the total job 22.5% faster (1.23× geometric mean) and test execution 50% faster (1.50× geometric mean). Lightpanda finishes faster on total wall-clock time in 7 of 10 runs and on test execution in 9 of 10 runs. Setup overhead dominates total duration for both engines, accounting for roughly 70% of total job time; Lightpanda's smaller binary reduces this phase by 20.6%. All three TodoMVC tests pass on both engines without modification.

These results are consistent with Stage 1: the speedup is real but moderate in the context of a full pipeline, where setup overhead dilutes the browser-level advantage. For a suite that runs hundreds of tests, the test execution speedup would compound more significantly.

C. Stage 3: Real Application Test

Stage 3 is the decisive experiment. The full 40-test Playwright suite is executed against `practicesoftwaretesting.com`: a publicly available Angular-based e-commerce application with authentication, product listings, pagination, filtering, search, cart management, and a contact form. This represents a realistic workload for a professional E2E test pipeline.

Google Chrome passes all 40 tests. Lightpanda fails 30 of 40 – a **75% failure rate**. Table III summarises results by functional category.

The failures are not caused by flaky tests or application-specific quirks; all 40 tests were verified to be stable and passing on Chrome before the Lightpanda run. The root causes, as observed from Playwright error output, fall into five distinct categories:

- **Isolated browser contexts not supported.** The test suite calls `browser.newContext()` before each test to create an isolated browsing session – the standard Playwright pattern for preventing state leakage between tests [2]. Lightpanda does not implement `newContext()`, causing every test that depends on a fresh context to fail immediately with a protocol error before any assertion runs. This alone accounts for the majority of failures.
- **CDP domains missing.** Playwright internally issues calls to CDP domains such as `Emulation`, `Network`, and `Input` to configure the browser environment and dispatch events. Several of these domains are absent or return incomplete responses in Lightpanda, producing protocol-level failures that surface as timeout errors or unresolvable locators.
- **JavaScript engine incompleteness.** The application under test is built with Angular, a framework whose runtime relies on a range of browser APIs including `MutationObserver`, custom event dispatch, and modern array and object methods. Lightpanda's JavaScript engine does not fully cover these APIs, causing the framework to fail to initialise correctly on several pages, which in turn means that Playwright's auto-waiting mechanism never sees the expected DOM state.
- **No locale or timezone support.** Playwright supports configuring browser context locale and timezone via `browser.newContext({locale, timezoneId})`. The test application renders its UI in German (`de-DE`), so tests use German-language role names and labels (e.g. `Email Adresse`, `Suche`, `Senden`). Lightpanda does not support locale configuration, which means these Playwright locators cannot be matched even when the DOM renders.
- **Confusing and misleading error messages.** Lightpanda's CDP error responses do not consistently identify which domain or method failed. This makes it substantially harder to diagnose and triage failures compared to Chrome, which provides structured protocol error payloads.

The authentication tests fare best (7/8 passing) because several involve only basic navigation to a login page and simple DOM visibility assertions – the narrowest slice of browser functionality. A handful of tests in other categories that pass are similarly limited: static visibility checks on the homepage that succeed because the initial page load partially renders before the JavaScript engine failures surface. No test involving dynamic interaction (clicking, selecting, form submission) or context isolation passes on Lightpanda.

V. EVALUATION AND DISCUSSION

A. Performance vs. Compatibility: An Irreconcilable Trade-off

The results across the three stages reveal a fundamental tension. Lightpanda delivers genuine and reproducible per-

formance benefits in synthetic settings: approximately 2× faster request handling and 10× lower memory in controlled micro-benchmarks, and a modest but real 1.23× total pipeline speedup in a simple CI workflow. These gains are consistent across hardware architectures (Intel x86-64 and Apple M3 Max) and arise from Lightpanda's lightweight single-process design, which avoids the rendering, GPU, and network service overhead of Chrome's multi-process architecture [5].

However, performance advantages are irrelevant if the engine cannot execute the test suite. Stage 3 demonstrates unambiguously that Lightpanda cannot serve as a drop-in replacement for Chrome Headless in real-world E2E testing: a 75% test failure rate on a standard e-commerce application renders the engine unusable for this purpose regardless of its speed.

B. The Core Compatibility Problem

The most critical missing capability is the absence of isolated browser contexts. The Playwright best practice of creating a fresh `BrowserContext` per test is not supported by Lightpanda, which means the standard test isolation pattern fails at the framework level before any test logic runs. This is not an edge case — it is the default and recommended way to structure Playwright test suites [2]. Until Lightpanda implements `browser.newContext()`, it is incompatible with the overwhelming majority of professionally structured Playwright suites.

The missing CDP domains and incomplete JavaScript engine coverage compound this problem. Angular applications, like many modern frameworks, rely on browser APIs (event dispatch, `MutationObserver`, layout APIs) that Lightpanda has not yet implemented. Playwright's own auto-waiting mechanism internally uses CDP calls that similarly depend on domain coverage Lightpanda does not provide.

C. Critique of Lightpanda's Published Benchmark

Lightpanda's headline performance claims — 9× faster execution, 16× less memory — deserve scrutiny in light of the Stage 3 results. These figures are derived from a "crawler benchmark" that navigates 933 pages of `demo-browser.lightpanda.io/amiibo/`: a purpose-built demo page consisting of a single image and a small number of internal links. This page requires no JavaScript framework, no authentication, no dynamic DOM mutation, and no complex CSS layout — precisely the features that expose Lightpanda's limitations in practice.

Claiming that this constitutes a benchmark across 933 "real web pages" is misleading. Real web applications, as demonstrated by Stage 3, rely on framework runtimes (Angular, React), client-side routing, dynamic content loading, form interactions, and authenticated sessions — none of which the Lightpanda demo page exercises. The benchmark has no ecological validity for E2E testing workloads, and the headline figures should not be used to evaluate Lightpanda's suitability as a Chrome replacement in production pipelines.

D. Where Lightpanda Is Applicable Today

The Stage 1 and Stage 2 results suggest a narrow but legitimate use case: web crawling and simple content extraction on statically rendered or minimally interactive pages. If a test suite is limited to basic navigation and DOM visibility assertions on simple pages — similar in complexity to the TodoMVC demo — and does not require browser context isolation, Lightpanda can provide a real speed and memory benefit. This corresponds roughly to what its benchmark actually measures, even if the marketing language overstates the scope.

For any team running a realistic E2E test suite against a modern web application, Lightpanda in its current state (v0.3.0, beta) is not viable. The Lightpanda GitHub issue tracker openly acknowledges this, listing partial CSS layout (incomplete Flexbox/Grid), limited WebSocket support, and numerous missing CDP features as known limitations of the current v0.x release series.

VI. Conclusion

This paper presented a three-stage empirical evaluation of Lightpanda as a drop-in replacement for Google Chrome Headless in Playwright-based end-to-end test pipelines. The headline finding is clear: **Lightpanda is not production-ready and cannot replace Chrome Headless for real-world E2E testing in its current form.**

The performance story is real but limited in scope. Lightpanda is approximately 2× faster and uses 10× less memory than Chrome in controlled benchmarks against simple pages, and delivers a 1.23× pipeline speedup in a three-test GitHub Actions workflow. These advantages stem from its lightweight single-process design and minimal rendering stack.

The compatibility story undoes the performance gains. Against a realistic Angular e-commerce application with 40 Playwright tests, Lightpanda fails 30 – a 75% failure rate driven by fundamental missing capabilities: no isolated browser context support, incomplete CDP domain coverage, an incomplete JavaScript engine, and absent locale handling. These are not minor gaps; they disqualify Lightpanda for the standard patterns used in professional E2E test suites.

Additionally, Lightpanda's advertised benchmark figures – 9× faster, 16× less memory across 933 "real web pages" – are derived from a trivial purpose-built demo page that bears no resemblance to real web applications. These figures cannot be used to assess Lightpanda's suitability for production E2E pipelines.

Lightpanda is a promising research-stage project for web crawling and simple navigation automation, but its current maturity level makes it unsuitable as a Chrome replacement for teams with non-trivial test suites. Future work should re-evaluate Lightpanda as browser context isolation, CDP coverage, and JavaScript engine support mature in subsequent releases.

REFERENCES

[1] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[2] Microsoft Corporation, "Playwright: Fast and reliable end-to-end testing for modern web apps." https://playwright.dev, 2024. Accessed: 2026.

[3] E. Laukkanen, J. Itkonen, and C. Lassenius, "Problems, causes and solutions when adopting continuous delivery—a systematic literature review," in *Information and Software Technology*, vol. 82, pp. 55–79, Elsevier, 2017.

[4] K. Duda and K. Kluza, "Web scraping and crawling are legal: When and how?," *Computer Law & Security Review*, vol. 34, no. 4, pp. 703–718, 2018.

[5] C. Reis and S. D. Gribble, "Isolating web programs in modern browser architectures," in *Proceedings of the 4th ACM European Conference on Computer Systems (EuroSys)*, pp. 219–232, 2009.

[6] Google, "Chrome devtools protocol." https://chromedevtools.github.io/devtools-protocol/, 2024. Accessed: 2026.

[7] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 426–437, 2016.